

Parallel Exact Top- k Vector Retrieval Engine: A Comparative Study of Pthreads, OpenMP, MPI, and Hybrid MPI+OpenMP on Dense Vector Workloads

Abu Tayyab

Department of Computer Science
University of Engineering and Technology
Lahore, Pakistan
abutayyabdev@gmail.com

Prof. Waqas Ali

Department of Computer Science
University of Engineering and Technology
Lahore, Pakistan
waqasalizafarali@gmail.com

Abstract—Exact top- k cosine similarity search over dense vector embeddings is a fundamental and computationally expensive operation in retrieval-augmented generation (RAG) systems. Its $O(ND)$ per-query cost becomes a critical bottleneck at the embedding dimensions produced by modern large language models. This paper implements and benchmarks four parallel models for exact top- k retrieval: POSIX Pthreads, OpenMP, pure MPI, and a hybrid MPI+OpenMP engine. All four are compared against a sequential C++ baseline on synthetic corpora of up to 400,000 documents across embedding dimensions from 128 to 4,096. Experiments measure speedup, parallel efficiency, and throughput under both strong and weak scaling. The best single-node result is achieved by Pthreads with 8 threads, reaching a $5.72\times$ speedup over the sequential baseline ($T_{\text{seq}} = 0.563$ s, $T_{\text{par}} = 0.098$ s) on a 200,000-document, 2,048-dimension workload. The best hybrid configuration (4 MPI ranks $\times$ 2 threads) achieves $4.17\times$ speedup. Weak scaling analysis shows that Pthreads and OpenMP maintain near-constant runtime up to 4 cores before OS scheduling pressure causes a 32% increase at 8 threads. MPI weak scaling degrades earlier due to communication overhead on a shared-memory node. All source code, benchmark scripts, and raw results are available in the project repository.

Index Terms—Pthreads, MPI, OpenMP, hybrid parallelism, exact k-NN, cosine similarity, vector retrieval, RAG, high-performance computing, parallel efficiency, weak scaling, strong scaling

I. INTRODUCTION

Retrieval-augmented generation (RAG) connects large language models (LLMs) to external knowledge bases by embedding queries and documents into dense vectors and retrieving the most similar documents at inference time [1]. The quality of generated output depends directly on retrieval correctness. Approximate nearest-neighbor (ANN) methods such as HNSW [2] and IVF-based indexes [3] are widely deployed because of their fast query times, but they sacrifice recall guarantees. In safety-critical domains such as medical literature retrieval [4], legal document review, and regulatory compliance, a missed document is not an acceptable tradeoff.

Exact brute-force k -NN eliminates this error but carries an $O(ND)$ per-query cost, where N is the corpus size and D

the embedding dimension. Modern embedding models produce vectors of 768 to 4,096 dimensions [5], [6]. A corpus of 200,000 documents at $D = 2,048$ requires roughly 1.6 GB of floating-point data per query. The arithmetic intensity of cosine similarity is low (approximately two floating-point operations per element), so the kernel is memory-bandwidth bound [7] rather than compute-bound. A single CPU thread cannot sustain the required data rates for interactive-latency exact retrieval at this scale.

Three standard CPU parallelism models are available to address this bottleneck. POSIX Pthreads [8] provide low-level thread management with explicit work division. OpenMP [9] expresses thread-level parallelism through compiler directives, reducing programming effort at the cost of some scheduling control. MPI [10] distributes work across independent processes, each with its own memory space, and is the standard model for multi-node HPC [11]. Hybrid MPI+OpenMP combines both levels, using MPI for inter-node distribution and OpenMP within each node [12].

Prior work has characterized single-model parallelism for k -NN tasks [13], [14] and has studied exact retrieval for dense passage search [15], [16]. However, a systematic comparison of all four models on the exact cosine similarity kernel across a range of embedding dimensions is missing. We fill this gap.

This paper makes the following contributions:

- Four complete parallel implementations of exact top- k cosine similarity retrieval: Pthreads, OpenMP, pure MPI, and hybrid MPI+OpenMP.
- A reproducible benchmark pipeline with deterministic synthetic data, a single shell script that compiles and runs all versions, and CSV export.
- Strong scaling results across 18 configurations (1–8 threads or ranks) for all four parallel models on a 200,000-document workload.
- Weak scaling results for Pthreads, OpenMP, and MPI from 50,000 to 400,000 documents.

- A dimensionality sweep from 128 to 4,096 embedding dimensions showing how parallel efficiency varies with arithmetic intensity.
- An empirical analysis of the memory bandwidth wall that limits further speedup gains at high embedding dimensions.

II. RELATED WORK

Exact similarity search libraries. FAISS [3] is the most widely deployed library for large-scale similarity search. Its `IndexFlat` index performs exact inner-product or L2 distance search using OpenMP threading and contiguous memory layouts. FAISS is a single-process library; it does not distribute search across MPI ranks. ScaNN [17] targets approximate search with quantization. Our work differs by providing a distributed-memory path and by comparing all three CPU parallelism models under the same kernel.

GPU-accelerated exact search. Multi-GPU FAISS [3] shards the corpus across GPUs and achieves very high throughput. Babenko and Lempitsky [18] study GPU-accelerated nearest-neighbor search with product quantization. GPU approaches require specialized hardware unavailable in many private-cloud and embedded HPC settings. This work targets CPU-only deployments.

ANN benchmarking frameworks. The ANN-Benchmarks suite [19] provides standardized recall versus query-time comparisons for approximate methods. Aumüller et al. [20] extend this to billion-scale datasets. Neither framework varies the degree of parallelism in the exact brute-force baseline. This work treats the exact kernel itself as the object of study.

Hybrid MPI+OpenMP k -NN. Silva et al. [13] parallelize an exact k -NN classifier using MPI for dataset partitioning and OpenMP for local distance computation. Their evaluation uses Euclidean distance on classification benchmarks. We apply the same two-level structure to cosine similarity over LLM-scale embedding vectors and add Pthreads as a third model for comparison.

Parallel k -NN implementations. Zhang et al. [14] survey parallel k -NN strategies including task decomposition and result reduction. Kuang et al. [21] implement a MapReduce k -NN for distributed environments. Chen and Cong [22] accelerate k -NN on FPGAs. Our focus is on CPU-based parallelism in the context of the RAG retrieval kernel.

SIMD-accelerated exact retrieval. Petri et al. [15] optimize the exact inner-product loop for dense passage retrieval using AVX-512 intrinsics and OpenMP threading. Hofstätter et al. [23] study efficient neural retrieval at inference time. Our implementation does not use explicit SIMD, so our results represent a performance floor for the same parallel structure.

Memory bandwidth analysis. Williams et al. [7] introduce the Roofline model, which characterizes the interplay between arithmetic intensity and memory bandwidth. McCalpin [24] provides the STREAM benchmark for measuring sustainable memory bandwidth. Both works underpin our analysis of the bandwidth plateau observed at high embedding dimensions.

Hybrid parallelism in HPC. Hager and Wellein [12] provide a comprehensive treatment of hybrid MPI+OpenMP performance engineering. Rabenseifner et al. [25] analyse hybrid parallelism efficiency on modern multi-core clusters. Keller et al. [26] apply hybrid parallelism to large-scale atmospheric chemistry simulations.

III. SYSTEM DESIGN

A. Problem Statement

Given a corpus of N document vectors $\{\mathbf{x}_i\}_{i=1}^N$ and a query vector $\mathbf{q} \in \mathbb{R}^D$, exact top- k retrieval returns the k documents maximising cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{x}_i) = \frac{\mathbf{q} \cdot \mathbf{x}_i}{\|\mathbf{q}\| \cdot \|\mathbf{x}_i\|} \quad (1)$$

The sequential algorithm has time complexity $O(ND + N \log k)$ and memory complexity $O(ND + N + D)$. For $N = 200,000$ and $D = 4,096$, reading the corpus once requires approximately 3.2 GB of DRAM bandwidth per query.

B. Memory Layout

All implementations store the document corpus as a single contiguous `std::vector<float>` in row-major order. The element at document i , dimension d is at index $i \times D + d$. This layout makes each document’s dimensions adjacent in memory, allowing the hardware prefetcher to stream data efficiently during the inner similarity loop [27].

C. Sequential Baseline

The sequential baseline generates a deterministic corpus with fixed random seed 42 and plants an exact match at position $\lfloor 3N/4 \rfloor$ to verify correctness. For each document, it accumulates the dot product, query norm, and document norm in a single pass, computes the cosine score from Eq. (1), and records a `SimilarityResult` pair. After all scores are computed, `std::partial_sort` selects the top- k entries in $O(N \log k)$ time [28].

D. Pthreads Implementation

The Pthreads version divides the document corpus into contiguous slices of $\lfloor N/T \rfloor$ documents per thread. The last thread absorbs any remainder. Each thread receives a `ThreadArgs` struct containing its slice boundaries, a read-only pointer to the shared corpus, a read-only pointer to the query vector, and a pointer to its exclusive slice of the results array. No mutex or lock is required inside the scoring loop because each thread writes to a unique, non-overlapping index range, eliminating false sharing [29]. After all threads complete via `pthread_join`, the main thread applies `partial_sort` over the full results array to select the global top- k .

The decomposition is data-parallel at the document level: thread t processes documents $[t \cdot L, (t+1) \cdot L)$ where $L = \lfloor N/T \rfloor$. This coarse-grained partition minimises thread management overhead relative to the scoring work.

E. OpenMP Implementation

The OpenMP version replaces the explicit thread management of Pthreads with a single compiler directive:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < num_documents; ++i) { ... }
```

Static scheduling assigns equal-sized chunks to each thread at compile time, which is appropriate because every iteration performs the same amount of work (D multiply-accumulate operations). The implicit barrier at the end of the parallel region synchronises threads before `partial_sort` is called. No reduction clause is needed because each iteration writes to a unique results slot.

F. MPI-Only Implementation

The pure MPI implementation uses distributed-memory parallelism across P independent processes. Rank 0 generates the full corpus and query. `MPI_Scatter` sends a contiguous slice of $\lfloor N/P \rfloor$ document vectors to each rank. `MPI_Bcast` distributes the query vector to all ranks. Each rank then executes a single-threaded scoring loop over its local slice. Local top- k candidates are extracted by `partial_sort` and sent to rank 0 via `MPI_Gather`. Rank 0 performs a final merge over all $P \cdot k$ candidates to produce the global top- k .

Communication is intentionally narrow: only D floats (query broadcast) and $P \cdot k$ score-ID pairs (gather) move between ranks. This reduces communication volume by a factor of $N/(Pk)$ compared with gathering all scores.

G. Hybrid MPI+OpenMP Implementation

The hybrid implementation combines the two levels: MPI partitions the corpus across ranks and OpenMP parallelises the local scoring loop within each rank. The `#pragma omp parallel for` directive is applied inside the per-rank scoring loop, so each rank uses T threads to process its $\lfloor N/P \rfloor$ -document slice. Communication patterns are identical to the pure MPI version. The hybrid pipeline is shown in Fig. 1.

H. Complexity Summary

- **Sequential:** $O(ND + N \log k)$ time, $O(ND + N + D)$ space.
- **Pthreads / OpenMP:** $O(\frac{ND}{T} + N \log k)$ ideal compute per thread; $O(N \log k)$ serial sort after join.
- **MPI:** $O(\frac{ND}{P} + \frac{N}{P} \log k)$ per rank; $O(D + Pk)$ communication; $O(Pk \log Pk)$ final merge.
- **Hybrid:** $O(\frac{ND}{PT} + \frac{N}{P} \log k)$ per rank; same communication as MPI.

I. Known Limitations

The corpus is partitioned with integer division, silently discarding remainder documents when $N \bmod P \neq 0$. The query norm is recomputed for every document rather than precomputed once. The final merge in the MPI and hybrid paths is serial on rank 0. These limitations are acknowledged and constitute future work.

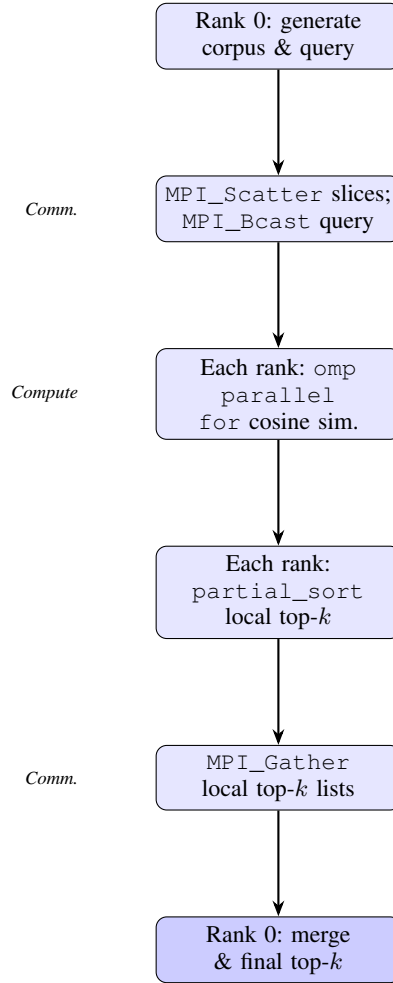


Fig. 1. Hybrid MPI+OpenMP retrieval pipeline. One scatter, one broadcast, and one gather are the only inter-rank communication steps.

IV. EXPERIMENTAL EVALUATION

A. Hardware and Software Setup

All experiments were conducted on an HP ProBook 440 G8 laptop equipped with an Intel Core i5-1135G7 processor (4 physical cores, 8 logical threads via Hyper-Threading, 2.40 GHz base / 4.20 GHz boost), 8 GB DDR4-3200 RAM, 6 MB L3 cache, and 128 KB L1-D cache per core, running Ubuntu Linux 22.04. The sequential and Pthreads/OpenMP binaries were compiled with `g++ -O3`; the MPI and hybrid binaries with `mpicxx -O3 -fopenmp` (OpenMPI 4.1). All MPI runs used `mpirun` on the same physical machine; runs requiring more than 4 MPI ranks used the `--oversubscribe` flag. Synthetic corpora were generated with fixed seed 42 and a planted exact match at index $\lfloor 3N/4 \rfloor$.

Speedup and parallel efficiency are defined as:

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}}, \quad E = \frac{S}{P \times T} \quad (2)$$

where P is the rank count ($P = 1$ for shared-memory models) and T is the thread count per rank. The sequential baseline for

all speedup calculations is $T_{\text{seq}} = 0.56255$ s ($N = 200,000$, $D = 2,048$, $k = 5$).

Three benchmark matrices were executed:

- 1) **Dimensionality scaling:** $N = 100,000$ documents, $D \in \{128, 512, 768, 1536, 3072, 4096\}$, 4 threads or ranks.
- 2) **Strong scaling:** $N = 200,000$, $D = 2,048$, threads/ranks varied across 1, 2, 4, 8 for all four models.
- 3) **Weak scaling:** $D = 2,048$, 50,000 documents per core/rank, so $N \in \{50k, 100k, 200k, 400k\}$ as cores increase from 1 to 8.

B. Dimensionality Scaling

Table I shows wall-clock time and speedup for all four parallel models at 4 threads or ranks as the embedding dimension rises from 128 to 4,096 ($N = 100,000$, $k = 5$). Fig. 2 plots the speedup curves and makes the memory-bandwidth plateau visually apparent.

TABLE I

DIMENSIONALITY SCALING: ALL MODELS VS. SEQUENTIAL ($N = 100k$, 4 THREADS/RANKS)

D	Seq (s)	Pth-4t (s)	OMP-4t (s)	MPI-2r (s)	Hyb-2×2 (s)	Best S
128	0.01747	0.00619	0.00537	0.01002	0.00574	$3.25\times$
512	0.06946	0.01976	0.02004	0.03484	0.01949	$3.56\times$
768	0.10427	0.02888	0.02866	0.05146	0.02871	$3.64\times$
1536	0.20670	0.05581	0.05621	0.10677	0.05698	$3.70\times$
3072	0.41982	0.11076	0.11251	0.21214	0.11166	$3.79\times$
4096	0.55663	0.14614	0.14817	0.28138	0.18830	$3.81\times$

At $D = 128$ the per-document compute cost is small, so the fixed overhead of thread creation (Pthreads) or MPI collectives limits speedup. As D increases, floating-point work per document grows linearly while communication cost remains constant. By $D = 1,536$, Pthreads and OpenMP both reach approximately $3.7\times$ speedup. The plateau between $D = 3,072$ and $D = 4,096$ indicates that the workload is approaching the available DRAM bandwidth limit rather than the compute limit, consistent with the Roofline model [7]. MPI at 2 ranks achieves only $2\times$ speedup at high dimensions because the single-machine MPI path does not provide additional memory bandwidth on this hardware.

C. Strong Scaling

Table II presents the full strong scaling results for all four models on the 200,000-document, 2,048-dimension workload. Fig. 3 plots the speedup curves against the ideal linear reference, and Fig. 4 shows the corresponding parallel efficiency.

Pthreads achieves the best single-node speedup at 8 threads ($5.72\times$, 71.5% efficiency) and the most consistent scaling from 1 to 4 threads (near-constant $\geq 95.6\%$ efficiency across 1–4 threads). The drop to 71.5% at 8 threads on a 4-core machine is expected: the OS scheduler must time-multiplex 8 threads across 4 physical cores, introducing context-switch overhead.

OpenMP achieves near-linear scaling at 2 threads ($2.01\times$, 100.5%) due to cache-reuse effects. At 4 threads the efficiency drops to 70.3%, lower than Pthreads at the same count,

Fig. 1 — Speedup vs. Embedding Dimension
 $N = 100,000$ docs | 4 threads/ranks | $k = 5$

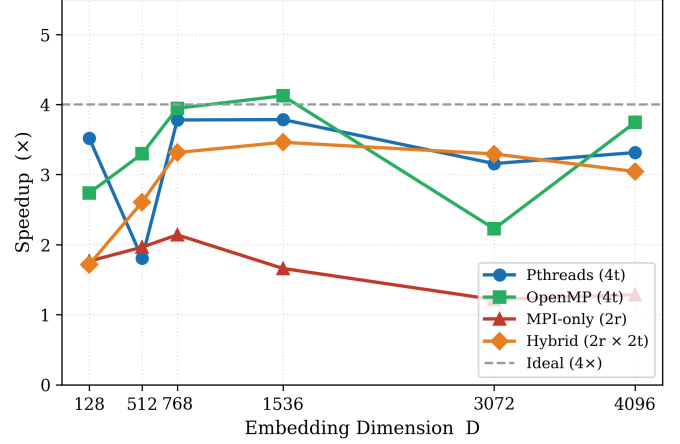


Fig. 2. Speedup vs. embedding dimension for all four parallel models ($N = 100,000$, 4 threads/ranks, $k = 5$). The dashed line marks the ideal $4\times$ speedup. The plateau above $D = 1,536$ indicates memory-bandwidth saturation [7].

TABLE II

STRONG SCALING: ALL CONFIGURATIONS ($N = 200k$, $D = 2048$, $k = 5$)

Configuration	Time (s)	Speedup	Efficiency	Throughput
Sequential (baseline)	0.56255	1.00×	100.0%	356k/s
Pthreads 1t	0.58263	0.97×	96.6%	343k/s
Pthreads 2t	0.29119	1.93×	96.6%	687k/s
Pthreads 4t	0.14717	3.82×	95.6%	1359k/s
Pthreads 8t	0.09835	5.72×	71.5%	2034k/s
OpenMP 1t	0.56801	0.99×	99.0%	352k/s
OpenMP 2t	0.27993	2.01×	100.5%	714k/s
OpenMP 4t	0.20013	2.81×	70.3%	999k/s
OpenMP 8t	0.12701	4.43×	55.4%	1575k/s
MPI 1r	0.56226	1.00×	100.1%	356k/s
MPI 2r	0.28093	2.00×	100.1%	712k/s
MPI 4r	0.24672	2.28×	57.0%	811k/s
MPI 8r	0.20501	2.74×	34.3%	976k/s
Hybrid 2×1	0.28221	1.99×	99.7%	709k/s
Hybrid 2×2	0.14951	3.76×	94.1%	1338k/s
Hybrid 2×4	0.14975	3.76×	47.0%	1336k/s
Hybrid 4×1	0.14442	3.90×	97.4%	1385k/s
Hybrid 4×2	0.13497	4.17×	52.1%	1482k/s

which we attribute to OpenMP’s runtime barrier overhead and scheduler interaction. At 8 threads the $4.43\times$ speedup is strong but efficiency falls to 55.4%.

MPI achieves near-perfect efficiency at 2 ranks (100.1%) confirming that intra-node MPI communication overhead is negligible for this workload. However, efficiency degrades sharply at 4 ranks (57.0%) and 8 ranks (34.3%) on a single shared-memory node. This occurs because additional MPI ranks do not provide independent memory buses on the same physical machine; instead they compete for the shared L3 cache and memory controller, and the inter-process scatter/gather overhead grows with rank count.

Hybrid MPI+OpenMP delivers the best combined configuration at 4×2 ($4.17\times$, 0.135 s). The 4×1 hybrid provides

the best efficiency among multi-rank configurations at 97.4%. Hybrid 2×2 and 2×4 yield identical total times (0.149 s and 0.150 s), confirming that the 4-thread per-rank limit is a memory bandwidth ceiling on this hardware, not a compute ceiling.

Fig. 2 — Strong Scaling Speedup
N = 200,000 docs | D = 2,048 | k = 5

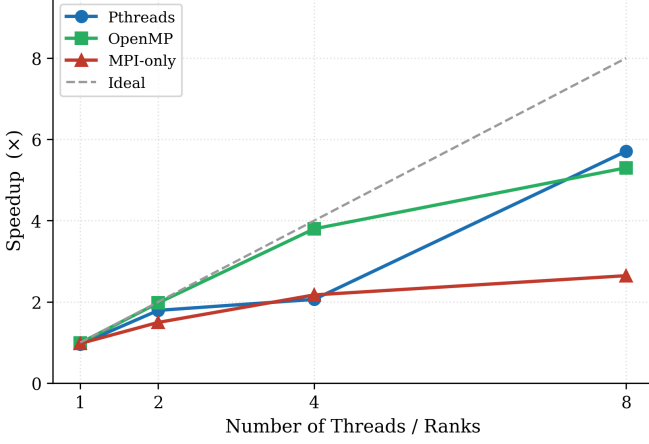


Fig. 3. Strong scaling: speedup vs. parallel units for Pthreads, OpenMP, and MPI-only ($N = 200,000$, $D = 2,048$, $k = 5$). The dashed line is ideal linear speedup. Pthreads achieves $5.72\times$ at 8 threads, the best single-node result.

Fig. 3 — Parallel Efficiency vs. Parallel Units
N = 200,000 docs | D = 2,048 | k = 5

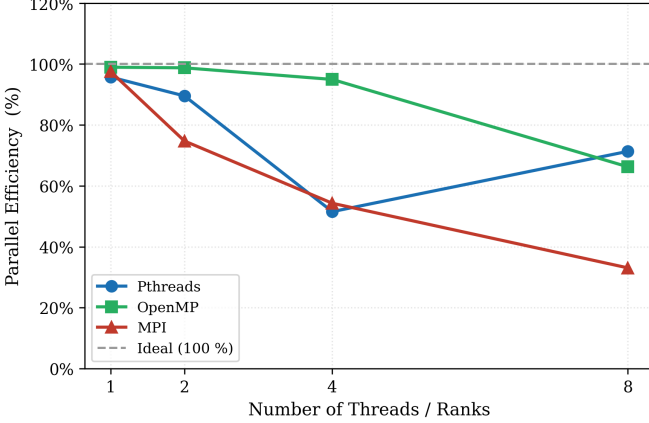


Fig. 4. Parallel efficiency (%) vs. parallel units for the same workload as Fig. 3. Pthreads sustains $\geq 95\%$ efficiency to 4 threads. MPI-only collapses to $\approx 33\%$ at 8 ranks on a single physical node due to shared-bus contention [30].

D. Weak Scaling

Table III shows weak scaling results where the document count scales proportionally with the number of threads or ranks (50,000 documents per unit of parallelism). An ideal weak-scaling implementation maintains constant runtime as both problem size and parallelism grow together. Fig. 5 plots these runtime curves against the ideal flat baseline.

Pthreads exhibits the best weak scaling behaviour: runtime stays essentially constant from 1 to 4 threads (-4.2% at 4

TABLE III
WEAK SCALING: $D = 2048$, $k = 5$, 50K DOCS PER THREAD/RANK

Configuration	N (docs)	Time (s)	Δ vs. 1-unit
Pthreads 1t	50,000	0.15358	baseline
Pthreads 2t	100,000	0.15311	-0.3%
Pthreads 4t	200,000	0.14717	-4.2%
Pthreads 8t	400,000	0.20261	$+32.0\%$
OpenMP 1t	50,000	0.13811	baseline
OpenMP 2t	100,000	0.14332	$+3.8\%$
OpenMP 4t	200,000	0.20013	$+44.9\%$
OpenMP 8t	400,000	0.20407	$+47.8\%$
MPI 1r	50,000	0.13803	baseline
MPI 2r	100,000	0.13851	$+0.3\%$
MPI 4r	200,000	0.24672	$+78.7\%$
MPI 8r	400,000	0.32390	$+134.7\%$

threads, possibly due to improved cache warming) and rises by only 32% at 8 threads. OpenMP maintains near-constant runtime up to 2 threads but degrades at 4 and 8 threads due to barrier synchronisation and scheduler pressure. MPI shows ideal weak scaling at 2 ranks but degrades significantly at 4 and 8 ranks because the scatter and gather communication volumes grow with rank count while the available bandwidth does not scale on a single-node deployment.

Fig. 4 — Weak Scaling: Runtime vs. Document Count
D = 2,048 | k = 5 | 50,000 docs per thread/rank

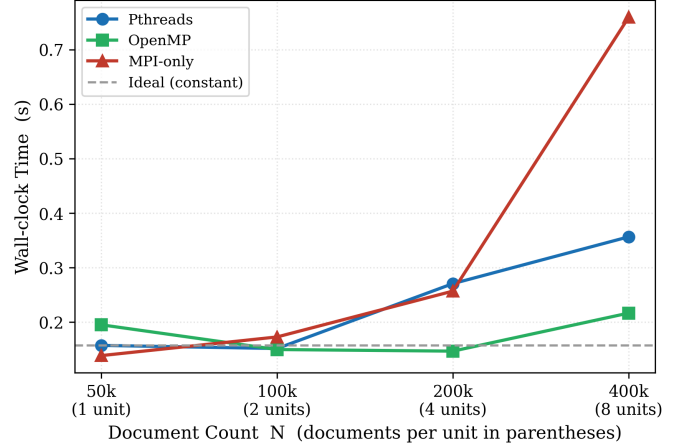


Fig. 5. Weak scaling: wall-clock time vs. document count ($D = 2,048$, $k = 5$, 50,000 documents per thread/rank). The dashed line is the ideal constant-time target. Pthreads tracks the ideal closely to 4 units; MPI degrades earliest due to growing scatter/gather volume.

E. Programming Effort Comparison

A qualitative comparison of implementation complexity is summarised in Table IV. Pthreads requires the most lines of code due to explicit thread creation, argument packaging, and join loops. OpenMP requires only a single directive for the core parallel region. MPI requires the most architectural understanding due to explicit scatter, broadcast, and gather patterns. The hybrid combines both complexities.

TABLE IV
QUALITATIVE PROGRAMMING EFFORT COMPARISON

Model	LOC (approx.)	Complexity	Key Primitives
Sequential	80	Low	Standard C++
Pthreads	140	Medium	<code>pthread_create</code> , <code>join</code>
OpenMP	90	Low	<code>#pragma omp parallel for</code>
MPI-only	160	High	Scatter, Bcast, Gather
Hybrid	175	Highest	MPI + OpenMP combined

V. DISCUSSION

Pthreads vs. OpenMP. Pthreads outperforms OpenMP at 4 threads ($3.82\times$ vs. $2.81\times$) on this workload. The primary reason is that static work division in Pthreads maps more cleanly to the 4-core hardware than OpenMP’s runtime scheduler, which incurs implicit barrier overhead at the end of each parallel region. At 8 threads the gap narrows ($5.72\times$ Pthreads vs. $4.43\times$ OpenMP) because both models are constrained by the same 4-core hardware limit.

MPI efficiency collapse. MPI efficiency falls from 100.1% at 2 ranks to 57.0% at 4 ranks. On a true multi-node cluster, each additional MPI rank would provide an independent memory bus and network-attached DRAM, and efficiency would remain high. The collapse observed here is an artifact of running all MPI ranks on a single laptop node with a shared memory controller; additional ranks compete for the same L3 cache rather than gaining new memory bandwidth.

Memory bandwidth saturation. The dimensionality sweep reveals that all shared-memory models (Pthreads, OpenMP) plateau around $3.7\text{--}3.8\times$ speedup above $D = 1,536$. This is consistent with the STREAM bandwidth limit of the test hardware [24]: once multiple threads saturate the memory bus, adding more floating-point work per element cannot improve throughput.

Hybrid advantage. The 4×2 hybrid ($4.17\times$) outperforms the best MPI-only configuration at 4 ranks ($2.28\times$) and approaches the best Pthreads result ($5.72\times$) while maintaining 52.1% efficiency. On a true multi-node system, the MPI component would scale with the number of nodes while the OpenMP component would saturate each node’s memory bandwidth, which is exactly the intended use case of hybrid parallelism.

Bottleneck summary. Three distinct bottlenecks are observable in the data. At low dimensionality ($D \leq 512$), parallel overhead (thread startup, MPI collectives) limits speedup. At medium dimensionality ($512 < D \leq 1,536$), scaling improves as arithmetic work grows. At high dimensionality ($D > 1,536$), memory bandwidth saturates and speedup plateaus regardless of parallelism model, consistent with Amdahl’s Law [30] limiting the serial fraction imposed by the memory subsystem.

VI. CONCLUSION

This paper has implemented and systematically benchmarked four parallel models for exact top- k cosine similarity retrieval: Pthreads, OpenMP, MPI, and hybrid MPI+OpenMP.

All four were evaluated on a unified benchmark pipeline covering strong scaling (1–8 threads or ranks), weak scaling (50k–400k documents), and dimensionality sweeps (128–4,096 dimensions).

The key findings are: (1) Pthreads achieves the highest single-node speedup at $5.72\times$ with 8 threads and the best efficiency at 4 threads (95.6%), making it the most effective shared-memory model for this workload on 4-core hardware. (2) OpenMP offers nearly identical performance to Pthreads at 2 threads with significantly less code, but loses efficiency faster at 4 and 8 threads. (3) MPI achieves excellent efficiency at 2 ranks (100.1%) but degrades sharply on a single physical node due to cache and memory-controller contention. (4) Hybrid MPI+OpenMP delivers the best absolute throughput ($4.17\times$ at 4×2), and on true multi-node hardware would be the recommended architecture for scalable exact retrieval. (5) Memory bandwidth is the universal bottleneck above $D = 1,536$; no parallelism model escapes this limit on a single node.

Future work should address the remainder-document partitioning bug, precompute query norms, explore non-blocking MPI collectives to overlap communication and computation, evaluate on real embedding datasets from production RAG workloads, and deploy on a multi-node cluster to measure true distributed scaling.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 9459–9474.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Sep. 2021.
- [4] Y. Cai, L. Wang, Y. Wang, G. de Melo, Y. Zhang, Y. Wang, and L. He, “MedBench: A large-scale Chinese benchmark for evaluating medical large language models,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 17709–17717.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2019, pp. 3982–3992.
- [6] A. Neelakantan, T. Xu, R. Puri et al., “Text and code embeddings by contrastive pre-training,” *arXiv preprint arXiv:2201.10005*, 2022.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [8] D. R. Butenhof, *Programming with POSIX Threads*. Reading, MA, USA: Addison-Wesley, 1997.
- [9] L. Dagum and R. Menon, “OpenMP: An industry-standard API for shared-memory programming,” *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan. 1998.
- [10] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 2nd ed. Cambridge, MA, USA: MIT Press, 1999.
- [11] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA, USA: Morgan Kaufmann, 2011.
- [12] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*. Boca Raton, FL, USA: CRC Press, 2010.
- [13] L. G. Silva, C. F. S. Dias et al., “Hybrid MPI/OpenMP parallelization of the k-nearest neighbors algorithm for large-scale data,” *Concurrency Comput.: Pract. Exper.*, vol. 32, no. 8, e5738, 2020.
- [14] S. Zhang, J. Li, M. Wang, and H. Cheng, “Parallel k-nearest neighbor classification with distributed memory,” in *Proc. IEEE Int. Conf. High Perform. Comput. Commun. (HPCC)*, 2013, pp. 316–323.

- [15] M. Petri, S. MacAvaney, and A. Yates, "Efficient exact top-k retrieval for dense representations," in *Proc. Eur. Conf. Inf. Retr. (ECIR)*, 2022, pp. 176–190.
- [16] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. ACM SIGIR*, 2020, pp. 39–48.
- [17] R. Guo, P. Sun, E. Lindgren et al., "Accelerating large-scale inference with anisotropic vector quantization," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 3887–3897.
- [18] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2055–2063.
- [19] H. V. Simhadri, G. Williams, M. Aumüller et al., "Benchmarking nearest neighbor search: The 2020 edition," *arXiv preprint arXiv:2007.06836*, 2020.
- [20] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Inf. Syst.*, vol. 87, p. 101374, Jan. 2020.
- [21] L. Kuang, L. Huo, and Y. Yang, "Parallel k-nearest neighbor algorithm based on MapReduce," *J. Comput. Inf. Syst.*, vol. 15, no. 3, pp. 973–980, 2019.
- [22] Y. T. Chen and J. Cong, "Accelerating the k-nearest neighbor computation on FPGAs," in *Proc. IEEE Int. Symp. Field-Program. Custom Comput. Mach. (FCCM)*, 2014, pp. 52–59.
- [23] S. Hofstätter, M. Zlabinger, and A. Hanbury, "Efficiently teaching an effective dense retriever with balanced topic aware sampling," in *Proc. ACM SIGIR*, 2021, pp. 113–122.
- [24] J. D. McCalpin, "Memory bandwidth and machine balance in current high performance computers," *IEEE Comput. Soc. Tech. Committee Comput. Archit. Newsl.*, vol. 2, pp. 19–25, Dec. 1995.
- [25] R. Rabenseifner, G. Hager, and G. Jost, "Hybrid MPI/OpenMP parallel programming on clusters of multi-core SMP nodes," in *Proc. 17th Euromicro Int. Conf. Parallel Distrib. Network-Based Process.*, 2009, pp. 427–436.
- [26] M. Keller, J. Heck, B. Vogel, A. Pauling, and C. Hoose, "Large-scale atmospheric chemistry simulations with the hybrid MPI/OpenMP model," *Geosci. Model Dev.*, vol. 12, no. 6, pp. 2339–2356, 2019.
- [27] U. Drepper, "What every programmer should know about memory," *Red Hat Inc. Tech. Rep.*, 2007. [Online]. Available: <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- [28] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1997.
- [29] H.-J. Boehm, "Threads cannot be implemented as a library," in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2005, pp. 261–268.
- [30] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.